Operating Systems Lab

Assignment 5: Process Synchronization using xv6

Name	Amar Kumar
Roll Number	2401MC51
Platform	xv6-riscv
Topic	Process Synchronization using xv6

1. Introduction

The objective of this assignment is to study and implement classical process synchronization problems in xv6-riscv. Four problems are implemented: Peterson's Algorithm, Producer-Consumer, Readers-Writers, and Dining Philosophers. The programs use shared memory and synchronization mechanisms to coordinate multiple processes and protect shared data.

2. Question 1: Peterson's Algorithm

Implementation: xv6 processes normally have separate address spaces, so a shared-memory page was added using a new `shm_get()` system call. The page stores `flag[2]`, `turn`, and `shared_counter`. A parent and child process execute Peterson's algorithm for 10 iterations.
Working: Before entering the critical section, a process sets its flag to indicate that it wants to enter and gives the other process the turn. It waits while the other process is interested and has the turn. After entering the critical section, it increments the shared counter and then clears its flag. Thus only one process enters the critical section at a time.
Result: Two processes perform 10 increments each, so the expected final value of `shared_counter` is 20.

3. Question 2: Producer-Consumer

Implementation: A circular buffer is stored in shared memory. Its default size is 5 and the program also accepts a smaller size as an argument. The producer generates values from 1 to 20, while the consumer removes and prints them.
Synchronization: Three semaphores are used. `EMPTY` counts the available buffer positions, `FULL` counts the produced items, and `MUTEX` provides exclusive access while modifying the buffer. The semaphores are implemented in the kernel using xv6 synchronization primitives and sleep/wakeup.
Result: The observed output showed the producer waiting when the buffer became full and the consumer waiting when it became empty. All values from 1 to 20 were produced and consumed in order, with no missing or duplicated item.

4. Question 3: Readers-Writers

Implementation: Three reader processes and two writer processes are created using `fork()`. The shared structure contains `data` and `read_count`.
Working: `RC_MUTEX` protects the reader count. The first reader locks the shared resource and the last reader releases it, allowing multiple readers to read the data concurrently. Writers acquire the resource exclusively, so no other reader or writer can access the shared data during a write. A `TURNSTILE` semaphore prevents new readers from continuously passing a waiting writer, reducing the possibility of indefinite writer starvation.

5. Question 4: Dining Philosophers

Implementation: Five philosophers are represented by five separate processes. There are five forks, with each fork represented by a binary semaphore. Each philosopher performs five cycles of thinking, becoming hungry, eating, and returning to thinking.
Deadlock avoidance: Resource ordering is used. A philosopher always acquires the lower-numbered fork first and the higher-numbered fork second. Because all philosophers follow the same ordering, a circular wait cannot form. After eating, the philosopher releases both forks.
Result: The program prints the state transitions and waits for all five philosopher processes to complete their cycles.

6. Files Added and Modified

Kernel files: `kernel/shm.c`, `kernel/semaphore.c`, `kernel/memlayout.h`, `kernel/defs.h`, `kernel/main.c`, `kernel/proc.c`, `kernel/syscall.h`, `kernel/syscall.c`, and `kernel/sysproc.c`.
User files: `user/peterson.c`, `user/prodcons.c`, `user/readwrite.c`, `user/dining.c`, `user/user.h`, and `user/usys.pl`.
Build file: `Makefile` was updated to include the new kernel object and user programs.

7. Build and Execution

The project is built from the xv6-riscv directory using `make clean` followed by `make`. The system is started with `make qemu`. The four programs are executed inside xv6 using `peterson`, `prodcons`, `readwrite`, and `dining`.

8. Conclusion

This assignment demonstrates how synchronization can be implemented at the operating-system level. Peterson's Algorithm provides mutual exclusion using shared variables, semaphores coordinate the bounded buffer, the Readers-Writers solution permits concurrent reading while keeping writers exclusive, and resource ordering prevents deadlock in Dining Philosophers. Together, the four programs demonstrate mutual exclusion, process coordination, blocking, concurrency, and deadlock avoidance in xv6.